

Servicio: flujo_pacientes.py

Archivo: `services/flujo_pacientes.py` · Documento técnico-pedagógico · Generado 2026-05-20 13:46

Este archivo gestiona las colas de pacientes en cada área del instituto (Triage, Laboratorio, Audiometría, Rayos X, Medicina Ocupacional). Es el motor que mueve a un paciente de un área a la siguiente según su protocolo.

Resumen general del cambio

Pasamos de un módulo aislado que solo guardaba nombres de pacientes en listas (con interacción por consola usando `input()`) a un servicio integrado que trabaja con objetos **Cita**, mueve pacientes entre áreas siguiendo el protocolo que se les asignó, registra timestamps por área y alimenta directamente al módulo de KPI.

ANTES	AHORA
<code>areas['Triage'].append('Juan')</code> (solo el nombre como texto)	<code>areas['Triage'].append(cita)</code> (el objeto Cita completo)
<code>tiempos = {'Triage': 5, 'Laboratorio': 10, 'Audiometría': 8, 'Rayos X': 12, 'Medicina Ocupacional': 15}</code> (valores fijos, sesgaban el cuello de botella)	<code>tiempo_espera = sum(random.randint(5, 20) for _ in range(posicion))</code> (aleatorio según cantidad en cola)
<code>registrar_paciente()</code> pedía <code>input()</code> en consola	<code>iniciar_flujo_desde_cita(cita)</code> recibe el objeto y lo coloca en su primera área
No existía <code>atender_paciente</code>	<code>atender_paciente(area, indice)</code> saca al paciente y lo mueve al siguiente área de su protocolo
No registraba ni hora ni qué cita se atendió	Registra timestamps (ingreso, atención) en <code>cita.tiempos_por_area</code>

1. Las colas pasaron de strings a objetos Cita

Antes del cambio: En el código original, cada área guardaba solo el nombre del paciente como una cadena de texto. Esto significaba que el flujo de pacientes era un módulo completamente desconectado: no sabía el DNI, ni la empresa, ni qué exámenes tenía pendientes esa persona.

Código original

```
areas = {
    "Triage": [],
    "Laboratorio": [],
    ...
}

def registrar_paciente():
    nombre = input("\nIngrese nombre del paciente: ").strip()
    areas[area].append(nombre)  # <-- solo guarda el string del nombre
```

Código actual

```
def iniciar_flujo_desde_cita(cita):
    # cita es un objeto Cita con paciente, protocolo, áreas pendientes...
    primer_area = cita.areas_pendientes[0]
    areas[primer_area].append(cita)  # <-- guarda el objeto completo
    cita.area_actual = primer_area
```

```

cita.estado = Cita.ESTADO_EN_FLUJO
cita.tiempos_por_area[primer_area] = {
    "ingreso": datetime.now(), "atencion": None, "espera_min": 0,
}
return primer_area

```

Para principiantes: Un **objeto** en Python es como una caja que guarda varias piezas de información juntas. La Cita tiene dentro: paciente, fecha, hora, protocolo, lista de áreas pendientes, etc. Guardar el objeto entero (en lugar de solo el nombre) significa que cuando atendemos al paciente, tenemos acceso a TODO lo que necesitamos saber de él, sin tener que buscarlo en otro lado.

2. Nueva función: atender_paciente(area, indice)

Antes no existía esta función. Ahora es la pieza central: cierra la atención de un paciente en un área y lo mueve a la siguiente del protocolo. Si era la última área, marca la cita como **Atendida**.

```

def atender_paciente(area, indice=0):
    # 1) Sacamos al paciente de la cola del área
    cita = areas[area].pop(indice)
    posicion_al_ingresar = (indice + 1)

    # 2) Calculamos el tiempo de espera realista
    tiempo_espera = sum(
        random.randint(TIEMPO_MIN_ATENCION, TIEMPO_MAX_ATENCION)
        for _ in range(posicion_al_ingresar)
    )

    # 3) Marcamos el área como completada para esta cita
    cita.areas_pendientes.remove(area)
    cita.areas_completadas.append(area)
    cita.tiempos_por_area[area]["atencion"] = datetime.now()
    cita.tiempos_por_area[area]["espera_min"] = tiempo_espera

    # 4) Avisamos al KPI para que registre la atención
    kpi.registrar_atencion(area, cita.paciente.nombre, tiempo_espera, cita=cita)

    # 5) Si quedan áreas pendientes, lo movemos a la siguiente.
    # Si no, la cita queda Atendida.
    if cita.areas_pendientes:
        siguiente_area = cita.areas_pendientes[0]
        areas[siguiente_area].append(cita)
        cita.area_actual = siguiente_area
        cita.tiempos_por_area[siguiente_area] = {...}
    else:
        cita.estado = Cita.ESTADO_ATENDIDA

```

Para principiantes: El parámetro *indice=0* permite atender al primer paciente de la cola por defecto, pero también a cualquier otro si pasas otro número. Esto se usa desde la UI cuando seleccionas un paciente específico de la lista.

3. Tiempos de espera ahora son aleatorios y realistas

Antes del cambio: Los tiempos fijos (Triage 5, Laboratorio 10, Medicina Ocupacional 15) hacían que Medicina Ocupacional SIEMPRE fuera el cuello de botella, porque $15 \times N$ personas es más alto que cualquier otra área. El Pareto siempre mostraba el mismo resultado.

```

TIEMPO_MIN_ATENCION = 5
TIEMPO_MAX_ATENCION = 20

```

```

# Tiempo realista: cada persona delante tarda entre 5 y 20 min de forma aleatoria.
# La espera total del paciente que se atiende es la suma de esos tiempos.
tiempo_espera = sum(
    random.randint(TIEMPO_MIN_ATENCION, TIEMPO_MAX_ATENCION)
    for _ in range(posicion_al_ingresar)
)

```

Para principiantes: `random.randint(5, 20)` devuelve un número entero al azar entre 5 y 20 (ambos incluidos). El `sum(... for _ in range(N))` es una forma corta de Python para sumar N números aleatorios. Esto simula lo que pasa en la realidad: no todas las consultas duran lo mismo.

4. Trazabilidad de tiempos por área

Cada paciente ahora lleva un diccionario interno con la hora de ingreso a cada área y la hora en que fue atendido. Esto es lo que permite generar el **timeline** visual y los reportes PDF de recorrido por paciente.

```
cita.tiempos_por_area[area] = {
    "ingreso": datetime.now(),    # cuándo entró a la cola del área
    "atencion": None,            # se llena cuando lo atienden
    "espera_min": 0,             # tiempo total de espera en el área
}
```

5. Funciones eliminadas o simplificadas

Las funciones `registrar_paciente()`, `verificar_saturacion()`, `ver_estado_areas()`, `ver_pacientes()` y `ver_estadisticas()` del código original usaban `input()` y `print()` y solo servían en modo consola. Las reemplazamos por **helpers de datos** (`estado_areas()`, `pacientes_en_area()`) que devuelven listas/diccionarios, sin tocar la consola. La interfaz gráfica los consume directamente.

Para principiantes: Esta separación se llama '**separar la lógica de la presentación**'. La lógica está en los servicios (qué hace el sistema) y la presentación está en la UI (cómo se muestra al usuario). Es una buena práctica porque permite cambiar una sin tocar la otra.